

INVESTIGACIÓN

Jetpack Compose

Desarrollo de app de promedio de notas

Autor

Oscar Alexander Quintanilla

Universidad

DON BOSCO

Asignatura

DESARROLLO DE APPS MOVILES DMS

Docente

ALEXANDER ALBERTO

SIGUENZA

Abril, 2026

Índice

1. Introducción
2. ¿Qué es Jetpack Compose?
 - 2.1 Origen y motivación
 - 2.2 Filosofía declarativa
3. Ventajas de Jetpack Compose
4. Decisiones de diseño
5. Conclusiones personales
6. Referencias

1. Introducción

El desarrollo de aplicaciones móviles para Android ha experimentado una transformación profunda durante los últimos años. Durante más de una década, la construcción de la interfaz de usuario en Android dependió de un sistema basado en archivos XML estáticos combinados con código Java o Kotlin que manipulaba imperativamente los componentes en tiempo de ejecución. Este modelo, aunque sólido, resultaba verboso, propenso a errores y difícil de mantener cuando las pantallas crecían en complejidad.

En el año 2021, Google publicó la versión estable de **Jetpack Compose**, un nuevo *toolkit* de UI declarativo que reemplaza el modelo XML por funciones de Kotlin que describen cómo debe verse la interfaz en cada momento. Inspirado en frameworks modernos como React, SwiftUI y Flutter, Compose pretende simplificar drásticamente la forma en que se construyen aplicaciones Android.

El presente documento tiene como objetivo investigar a fondo qué es Jetpack Compose, cuáles son sus ventajas frente al sistema tradicional, y cómo se aplican sus principios en el desarrollo de una aplicación real: una calculadora de promedio académico que permite a un estudiante registrarse, ingresar las notas de sus materias y conocer si aprobó o reprobó. Adicionalmente, se documentan las decisiones de diseño tomadas durante la construcción del proyecto y se ofrece una reflexión personal sobre la experiencia de uso de esta tecnología.

El público objetivo de este documento son estudiantes y desarrolladores Android que tengan conocimientos básicos de Kotlin pero aún no hayan adoptado Compose, así como cualquier lector interesado en comprender por qué la industria está migrando hacia paradigmas declarativos para la construcción de interfaces.

«La interfaz de usuario debería ser una función de su estado.» — Esta frase, repetida frecuentemente en la documentación oficial de Compose, resume el cambio mental que el desarrollador debe hacer al pasar del paradigma imperativo al declarativo, y es la idea que guía todo este trabajo.

2. ¿Qué es Jetpack Compose?

Jetpack Compose es el *toolkit* moderno y recomendado por Google para construir interfaces de usuario nativas en Android. Se basa en un paradigma declarativo: en lugar de crear y mutar manualmente objetos como `TextView` o `Button`, el desarrollador escribe funciones componibles (`@Composable`) que describen cómo debe lucir la pantalla en función de un estado dado.

Compose forma parte de la familia **Android Jetpack**: un conjunto de librerías modulares creadas para acelerar el desarrollo, fomentar buenas prácticas y reducir la fragmentación entre versiones del sistema operativo. Aunque está construido encima de la plataforma Android existente, sustituye por completo al sistema de *Views XML* cuando se desea una experiencia 100 % Compose.

2.1 Origen y motivación

El sistema clásico de *Views* de Android nació junto con la primera versión del sistema operativo en 2008. A lo largo de los años se le añadieron capas: `Fragments`, `RecyclerView`, `DataBinding`, `ViewBinding`, `LiveData`, etc. El resultado era un ecosistema potente pero fragmentado, con código repetitivo (*boilerplate*) y un costo de aprendizaje elevado.

En paralelo, otras plataformas adoptaron paradigmas declarativos que demostraron una productividad notablemente superior: React (2013) en la web, Flutter (2017) para apps multiplataforma y SwiftUI (2019) en iOS. Google reconoció que Android necesitaba modernizarse y, tras varios años de desarrollo, en julio de 2021 lanzó la versión 1.0 estable de Jetpack Compose.

2.2 Filosofía declarativa

La idea central detrás de Compose puede resumirse en una sola fórmula:

$$UI = f(\text{estado})$$

Esto significa que la interfaz no es un objeto que se modifica paso a paso, sino el resultado de aplicar una función pura sobre un conjunto de datos. Cuando el estado cambia, Compose vuelve a ejecutar las funciones afectadas (un proceso llamado *recomposición*) y actualiza únicamente las partes de la pantalla que lo necesitan. El desarrollador deja de preocuparse por sincronizar manualmente los componentes con sus datos.

Como contraste, en el modelo imperativo tradicional el desarrollador tenía que escribir explícitamente cosas como `textView.setText("Hola")` cada vez que un dato cambiaba; con Compose, simplemente se escribe `Text(saludo)` y se garantiza que el texto siempre refleja el valor actual de `saludo`.

3. Ventajas de Jetpack Compose

Jetpack Compose ofrece beneficios concretos que justifican su adopción. A continuación se describen los más relevantes desde la perspectiva del desarrollador y del proyecto.

- **Menos código.** Compose elimina archivos XML, adaptadores manuales para listas, `findViewById`, `ViewBinding` y gran parte del boilerplate. Una pantalla típica que en XML+Kotlin requiere 200 líneas se construye en Compose con 60–80 líneas, todas en un único archivo Kotlin.
- **Sintaxis declarativa intuitiva.** Las funciones componibles describen el «qué» y no el «cómo»: el motor se encarga de actualizar la UI cuando los datos cambian. Esto reduce errores comunes como olvidar actualizar una vista al modificar un dato.
- **Integración con Kotlin.** Compose explota al máximo las características modernas del lenguaje: lambdas, `data classes`, *extension functions*, `flow`, corrutinas, *scoped functions*, etc. Esto produce código expresivo y seguro a nivel de tipos.
- **Material 3 listo para usar.** Compose incluye una implementación oficial de Material Design 3 lista para producción, con soporte de *Dynamic Color*, modo oscuro y temas personalizables sin tener que definir XML de estilos.
- **Vista previa instantánea (@Preview).** Cualquier `@Composable` puede previsualizarse en Android Studio sin necesidad de ejecutar la app, reduciendo enormemente el ciclo de iteración visual.
- **Animaciones simplificadas.** Animar valores numéricos, colores o tamaños es tan simple como `animateColorAsState(...)`; las transiciones complejas que antes requerían `AnimatorSet` se vuelven triviales.
- **Interoperabilidad con el sistema antiguo.** Una app puede mezclar pantallas Compose con pantallas XML tradicionales mediante `ComposeView` y `AndroidView`, lo que permite migraciones graduales en proyectos grandes.
- **Mejor rendimiento en muchos casos.** El sistema de recomposición inteligente evita redibujar componentes que no cambiaron, y elimina costosos inflados de XML.
- **Estandarización de la arquitectura.** Compose se complementa naturalmente con MVVM, `StateFlow`, Hilt, Room y Navigation Compose, formando un stack moderno y coherente.
- **Comunidad y respaldo de Google.** Es la dirección oficial de Google para el desarrollo Android. Toda la documentación nueva, *codelabs*, charlas y librerías modernas asumen Compose como punto de partida.

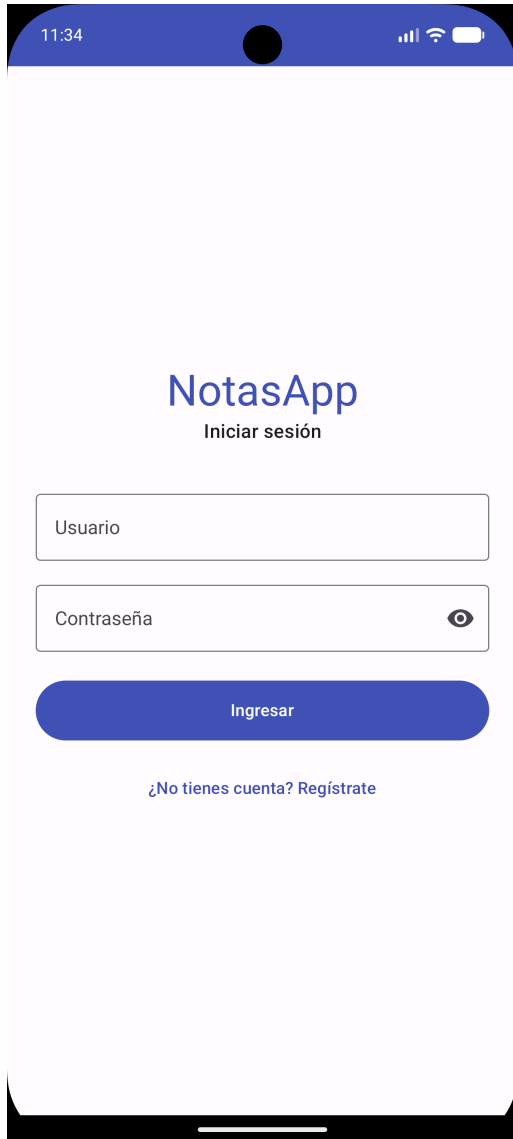
4. Decisiones de diseño

Construir el proyecto requirió tomar varias decisiones técnicas. A continuación se documentan las más importantes junto con su justificación.

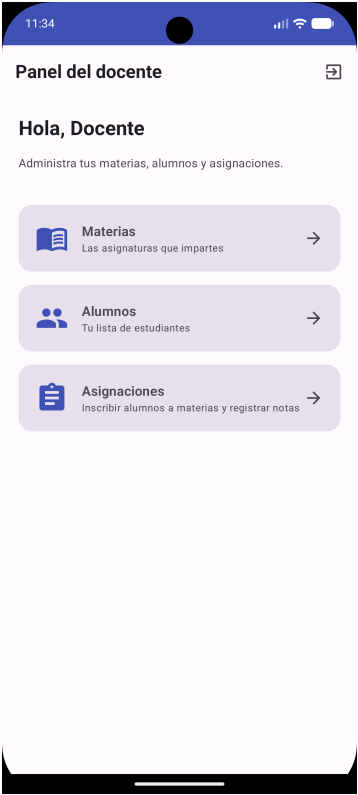
- **Persistencia local con Room en lugar de Firebase.** Se escogió Room porque (1) cumple el alcance académico sin depender de servicios externos, (2) no requiere cuenta de Google Cloud ni configuración de claves, y (3) muestra cómo Compose se integra con corrutinas y `Flow` para una UI reactiva. Firebase quedó como evolución futura.
- **Lista dinámica de materias en lugar de número fijo.** Una lista dinámica refleja mejor la realidad universitaria, donde cada estudiante cursa un número distinto de asignaturas. Además permite practicar el componente `LazyColumn`, central en Compose.
- **Cálculo del promedio como media aritmética simple.** Se priorizó la claridad y la facilidad de uso. La estructura del código permite extender fácilmente a un esquema de cortes ponderados sin reescribir la UI.
- **Escala 0 – 10 con aprobación en 6.0.** Los valores son constantes públicas en el `NotasViewModel` (`NOTA_APROBATORIA`, `NOTA_MAXIMA`, `NOTA_MINIMA`), de modo que ajustar la escala a otro sistema (p. ej. 0–100 con corte en 70) requiere cambiar una sola línea.
- **Almacenamiento de la contraseña en texto plano.** Para cumplir el alcance del proyecto sin introducir complejidad criptográfica, la contraseña se guarda directamente. En una aplicación real debería usarse `BCrypt`, `Argon2` o `EncryptedSharedPreferences`, lo cual queda explícitamente advertido en el código fuente.
- **Navegación basada en Navigation Compose.** Es la solución oficial y se integra de forma natural con el resto del stack. Permite definir argumentos tipados y manejar la pila con `popUpTo` e `inclusive`, fundamental para que «cerrar sesión» limpie completamente la historia.
- **UI con Material 3.** Aporta un look moderno sin esfuerzo, soporte de modo oscuro automático y componentes accesibles desde fábrica.
- **Validación temprana en la UI.** Las pantallas validan que los campos no estén vacíos y que la nota esté entre 0 y 10 antes de delegar al `ViewModel`. Esto provee retroalimentación inmediata al usuario.
- **Una sola Activity.** Toda la app vive dentro de `MainActivity`. Esto simplifica el ciclo de vida y es la práctica recomendada actualmente para apps Compose.

IMÁGENES DEL APLICATIVO

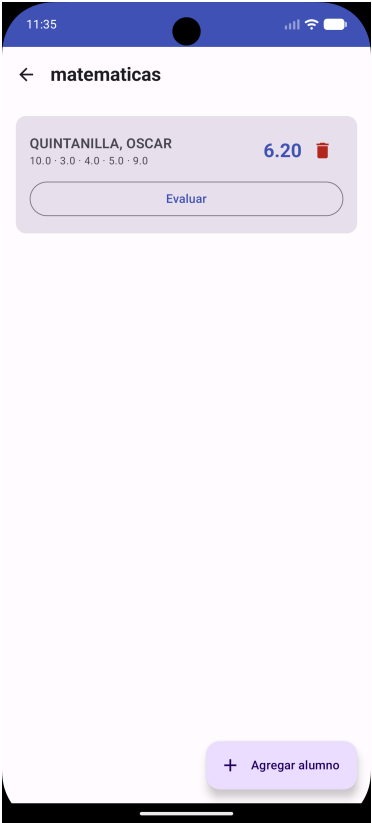
Login del aplicativo



MENU PRINCIPAL DE USUARIO



APARTADO DEL CRUD DE ALUMNOS



APARTADO DE LAS NOTAS

11:36

←

Evaluar

Materia

matematicas

Alumno

QUINTANILLA, OSCAR

Notas (0 a 10). Puedes dejar vacías las que aún no tengas.

Nota 1

10.00

Nota 2

3.00

Nota 3

4.00

Nota 4

5.00

Nota 5

9.00

Promedio (en vivo)

6.20

Calculado con 5 de 5 notas.

Guardar notas

5. Conclusiones personales

Tras investigar la documentación oficial, completar varios *code labs* y construir esta aplicación desde cero, mi percepción sobre Jetpack Compose es ampliamente positiva. Las siguientes son las conclusiones más significativas que extraigo de la experiencia:

Productividad real, no marketing

El primer impacto al programar en Compose es la enorme reducción de archivos: una pantalla que en el sistema XML necesitaría un `layout.xml`, un `Fragment.kt`, un `Adapter.kt` y un `ViewHolder.kt` aquí cabe en un único archivo Kotlin de menos de cien líneas. Esto no sólo se traduce en menos código, sino en menos puntos donde algo puede salir mal.

La curva de aprendizaje es moderada

Si el desarrollador ya está familiarizado con Kotlin y entiende qué es una *lambda*, la transición a Compose es relativamente suave. El concepto de recomposición y el flujo unidireccional toman algo de tiempo en interiorizarse, pero son consistentes con la forma moderna de pensar la UI.

La interoperabilidad es un puente seguro

Saber que se puede convivir con código XML existente convierte a Compose en una opción viable incluso para proyectos legacy: no es necesario reescribir todo de un día para otro. Esta es una decisión de Google que reduce el riesgo de adopción para empresas.

Algunos puntos a mejorar

El compilador de Compose puede ser estricto y los mensajes de error a veces son extensos. Además, ciertos componentes maduros del mundo XML (como mapas, vistas de calendario complejas o gráficas avanzadas) aún tienen mejor soporte fuera de Compose, lo que obliga a usar `AndroidView` como puente. Sin embargo, la comunidad y el ecosistema crecen rápidamente y estos huecos se van cerrando con cada release.

Sobre la aplicación construida

Construir **NotasApp** permitió integrar todo el stack moderno: Compose para la UI, Room para persistencia, Coroutines y `Flow` para la asincronía, ViewModel para sobrevivir a cambios de configuración y Navigation Compose para coordinar las pantallas. Cada elección encajó con la siguiente sin esfuerzo, lo cual confirma que Compose no es una pieza aislada sino la columna vertebral de una arquitectura coherente.

Recomendación final

Para cualquier estudiante que empiece hoy a desarrollar para Android, la recomendación es clara: **aprender directamente Jetpack Compose**. Invertir tiempo en dominar el sistema XML clásico ya no aporta tanto valor pedagógico, dado que la industria está migrando decididamente hacia el paradigma declarativo.

6. Referencias

- Google. (2024). *Jetpack Compose documentation*. <https://developer.android.com/jetpack/compose>
- Google. (2024). *Thinking in Compose*. <https://developer.android.com/jetpack/compose/mental-model>
- Google. (2024). *State and Jetpack Compose*. <https://developer.android.com/jetpack/compose/state>
- Google. (2024). *Navigation in Compose*. <https://developer.android.com/jetpack/compose/navigation>
- Google. (2024). *Save data in a local database using Room*. <https://developer.android.com/training/data-storage/room>
- Google. (2023). *Guide to app architecture*. <https://developer.android.com/topic/architecture>
- Material Design. (2024). *Material 3 Components for Compose*. <https://m3.material.io/develop/android/jetpack-compose>
- Kotlin Foundation. (2024). *Kotlin Coroutines guide*. <https://kotlinlang.org/docs/coroutines-guide.html>